DATA SCIENCE - 22DS102006

5. **Perform Data Visualization with Matplotlib and SeaBorn considering online dataset for processing.**
    a. **Create a Line Plot by setting the title, axis labels, ticks, ticklabels , annotations on subplots and save to a file.**
    b. **Create Bar Plots using Series and DataFrame index.**
        i. **Create bar plots with a DataFrame to group the values in each row together in a group in bars side by side for each value.**
        ii. **Create stacked bar plots from a DataFrame.**
    c. **Create Histogram to display the value frequency and Density Plot to generate continuous probability distribution function for observed data.**
    d. **Create Scatter Plot and examine the relationship between two one-dimensional data series.**
    e. **Create Box plots to visualize data with many categorical variables.**

Aim

To perform **data visualization using Matplotlib and Seaborn** by loading an online dataset, processing the data, and representing it using different types of plots such as **line plot, bar plot, histogram, and scatter plot**.

Software Requirements

- Python 3.x
- Pandas
- Matplotlib
- Seaborn
- Jupyter Notebook / Google Colab / Python IDE

Syntax

Read an online CSV dataset

```
pd.read_csv("URL")
```

Matplotlib plot

```
plt.plot(x, y)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Title")
plt.show()
```

Seaborn plot

```
sns.scatterplot(data=df, x="column1", y="column2")
```

Solution Program

The following program uses the **Iris dataset available online**.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load online dataset
url =
"https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv
"

df = pd.read_csv(url)

# Display first five records
print("First Five Records:")
print(df.head())

# Display dataset information
print("\nDataset Information:")
print(df.info())

# Display statistical summary
print("\nStatistical Summary:")
print(df.describe())

# ------------------------------------------------
# 1. Matplotlib - Line Plot
# ------------------------------------------------

plt.figure(figsize=(8, 5))

plt.plot(df.index, df["sepal_length"], label="Sepal Length")
plt.plot(df.index, df["petal_length"], label="Petal Length")

plt.xlabel("Sample Index")
plt.ylabel("Length")
plt.title("Sepal Length and Petal Length")
plt.legend()
plt.grid()
plt.show()


# ------------------------------------------------
# 2. Seaborn - Scatter Plot
# ------------------------------------------------

plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x="sepal_length",
    y="petal_length",
    hue="species"
```

```
)

plt.title("Sepal Length vs Petal Length")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")
plt.show()


# ------------------------------------------------
# 3. Seaborn - Histogram
# ------------------------------------------------

plt.figure(figsize=(8, 5))

sns.histplot(
    data=df,
    x="sepal_length",
    hue="species",
    kde=True
)

plt.title("Distribution of Sepal Length")
plt.xlabel("Sepal Length")
plt.ylabel("Frequency")
plt.show()


# ------------------------------------------------
# 4. Seaborn - Box Plot
# ------------------------------------------------

plt.figure(figsize=(8, 5))

sns.boxplot(
    data=df,
    x="species",
    y="petal_length"
)

plt.title("Petal Length by Species")
plt.xlabel("Species")
plt.ylabel("Petal Length")
plt.show()


# ------------------------------------------------
# 5. Seaborn - Pair Plot
# ------------------------------------------------

sns.pairplot(
    df,
    hue="species"
)

plt.show()
```

Expected Output

1. Dataset Preview

```
First Five Records:

   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
```

2. Statistical Summary

```
       sepal_length  sepal_width  petal_length  petal_width
count    150.000000   150.000000    150.000000   150.000000
mean       5.843333     3.057333      3.758000     1.199333
std        0.828066     0.435866      1.765298     0.762238
min        4.300000     2.000000      1.000000     0.100000
max        7.900000     4.400000      6.900000     2.500000
```

3. Visualizations

The program produces:

- **Line Plot:** Sepal length and petal length across the observations.
- **Scatter Plot:** Relationship between sepal length and petal length, grouped by species.
- **Histogram:** Distribution of sepal length.
- **Box Plot:** Comparison of petal length among different species.
- **Pair Plot:** Relationships among the numerical features of the Iris dataset.

Result

The online Iris dataset was successfully loaded and processed using **Pandas**. Different data visualizations were successfully created using **Matplotlib and Seaborn** to understand distributions, relationships, and differences among the dataset variables.

Create a Line Plot with Title, Axis Labels, Ticks, Tick Labels, Annotations and Save to a File

Aim

To create a **line plot using Matplotlib** by setting the title, axis labels, ticks, tick labels, and annotations on subplots, and save the generated plot to an image file.

Syntax

Create subplots

```
fig, ax = plt.subplots()
```

Set title

```
ax.set_title("Title")
```

Set axis labels

```
ax.set_xlabel("X-axis Label")
ax.set_ylabel("Y-axis Label")
```

Set ticks and tick labels

```
ax.set_xticks(x)
ax.set_xticklabels(labels)
```

Add annotation

```
ax.annotate("Text", xy=(x, y), xytext=(x1, y1),
            arrowprops=dict(arrowstyle="->"))
```

Save plot to a file

```
plt.savefig("filename.png", dpi=300, bbox_inches="tight")
```

Solution Program

```
import matplotlib.pyplot as plt

# Data
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [120, 150, 180, 160, 220, 250]
expenses = [80, 100, 120, 110, 140, 160]

# Create subplots
fig, ax = plt.subplots(2, 1, figsize=(10, 8))

# -------------------------------------------------
# Subplot 1: Sales
# -------------------------------------------------

ax[0].plot(
    months,
    sales,
    marker="o",
```

```
    linewidth=2,
    label="Sales"
)

ax[0].set_title("Monthly Sales")
ax[0].set_xlabel("Month")
ax[0].set_ylabel("Sales")

# Set ticks and tick labels
ax[0].set_xticks(range(len(months)))
ax[0].set_xticklabels(months)

# Annotation
ax[0].annotate(
    "Highest Sales",
    xy=(5, 250),
    xytext=(3.5, 270),
    arrowprops=dict(arrowstyle="->")
)

ax[0].legend()
ax[0].grid(True)


# ------------------------------------------------
# Subplot 2: Expenses
# ------------------------------------------------

ax[1].plot(
    months,
    expenses,
    marker="s",
    linewidth=2,
    label="Expenses"
)

ax[1].set_title("Monthly Expenses")
ax[1].set_xlabel("Month")
ax[1].set_ylabel("Expenses")

# Set ticks and tick labels
ax[1].set_xticks(range(len(months)))
ax[1].set_xticklabels(months)

# Annotation
ax[1].annotate(
    "Highest Expense",
    xy=(5, 160),
    xytext=(3.5, 180),
    arrowprops=dict(arrowstyle="->")
)

ax[1].legend()
ax[1].grid(True)

# Overall title
fig.suptitle("Monthly Sales and Expenses Analysis", fontsize=16)
```

```
# Adjust layout
plt.tight_layout()

# Save the plot to a file
plt.savefig(
    "monthly_sales_expenses.png",
    dpi=300,
    bbox_inches="tight"
)

# Display the plot
plt.show()
```

Expected Output

The program generates **two subplots**:

```
       Monthly Sales and Expenses Analysis

          Monthly Sales
Sales
250 |                          ● ← Highest Sales
220 |                    ●
180 |              ●
150 |         ●
120 |    ●
    +--------------------------------
       Jan  Feb  Mar  Apr  May  Jun
                   Month


          Monthly Expenses
Expenses
160 |                          ■ ← Highest Expense
140 |                    ■
120 |              ■
110 |         ■
100 |    ■
 80 | ●
    +--------------------------------
       Jan  Feb  Mar  Apr  May  Jun
                   Month
```

The actual Matplotlib output will display the plots graphically with **titles, axis labels, customized ticks, tick labels, legends, grids, and arrow annotations**.

Output File

The plot is saved as:

```
monthly_sales_expenses.png
```

at **300 DPI**, making it suitable for reports and practical records.

Result

Thus, a line plot with **titles, axis labels, ticks, tick labels, annotations, and multiple subplots** was successfully created using Matplotlib and saved to an image file.

B). Create Bar Plots using Series and DataFrame index.

iii. Create bar plots with a DataFrame to group the values in each row together in a group in bars side by side for each value.
iv. Create stacked bar plots from a DataFrame.

Aim

To create **bar plots using Pandas Series and DataFrame index**, and to create both **grouped bar plots** and **stacked bar plots** using Matplotlib.

---

i. Create Bar Plots with a DataFrame — Side-by-Side Bars

Syntax

```
df.plot(kind='bar')
```

or

```
df.plot.bar()
```

For a horizontal bar plot:

```
df.plot(kind='barh')
```

Solution Program

```
import pandas as pd
import matplotlib.pyplot as plt

# Create DataFrame
data = {
    'Python': [85, 90, 78, 88],
    'Java': [75, 82, 80, 85],
    'C++': [80, 76, 85, 90]
}

df = pd.DataFrame(
    data,
    index=['Student1', 'Student2', 'Student3', 'Student4']
)

print("DataFrame:")
print(df)

# Create grouped bar plot
df.plot(
    kind='bar',
    figsize=(8, 5)
)
```

```
plt.title("Student Marks in Different Programming Languages")
plt.xlabel("Students")
plt.ylabel("Marks")
plt.xticks(rotation=0)
plt.legend(title="Subjects")
plt.tight_layout()
plt.show()
```

Output

```
DataFrame:

          Python  Java  C++
Student1      85    75   80
Student2      90    82   76
Student3      78    80   85
Student4      88    85   90
```

The graphical output contains **three bars side by side for each student**:

- Python
- Java
- C++

For example:



Each row of the DataFrame is represented as a group of bars.

---

ii. Create Stacked Bar Plots from a DataFrame

Aim

To create a **stacked bar plot from a DataFrame**, where the values of different columns are stacked on top of each other for each index value.

Syntax

```
df.plot(kind='bar', stacked=True)
```

or

```
df.plot.bar(stacked=True)
```

Solution Program

```
import pandas as pd
import matplotlib.pyplot as plt

# Create DataFrame
data = {
    'Python': [85, 90, 78, 88],
    'Java': [75, 82, 80, 85],
    'C++': [80, 76, 85, 90]
}

df = pd.DataFrame(
    data,
    index=['Student1', 'Student2', 'Student3', 'Student4']
)

print("DataFrame:")
print(df)

# Create stacked bar plot
df.plot(
    kind='bar',
    stacked=True,
    figsize=(8, 5)
)

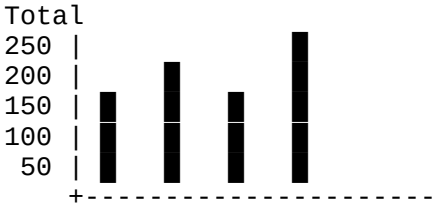
plt.title("Stacked Bar Plot of Student Marks")
plt.xlabel("Students")
plt.ylabel("Total Marks")
plt.xticks(rotation=0)
plt.legend(title="Subjects")
plt.tight_layout()
plt.show()
```

Output

```
DataFrame:

          Python  Java  C++
Student1      85    75   80
Student2      90    82   76
Student3      78    80   85
Student4      88    85   90
```

The graphical output contains **one bar for each student**, with Python, Java, and C++ marks stacked vertically.



```
      S1  S2  S3  S4
```

Difference Between Grouped and Stacked Bar Plots

Feature	Grouped Bar Plot	Stacked Bar Plot
Bars	Side by side	On top of each other
Code	`df.plot(kind='bar')`	`df.plot(kind='bar', stacked=True)`
Comparison	Easy to compare individual values	Easy to see total and contribution
Number of bars	Multiple bars per index	One combined bar per index

Result

Thus, **grouped bar plots** and **stacked bar plots** were successfully created from a Pandas DataFrame using its index values.

C). **Create Histogram to display the value frequency and Density Plot to generate continuous probability distribution function for observed data.**

Aim

To create a **Histogram** for displaying the frequency distribution of observed data and a **Density Plot** for representing the continuous probability distribution of the data using Python, Pandas, Matplotlib, and Seaborn.

Syntax

Histogram:

```
sns.histplot(data, bins=10, kde=False)
```

Density Plot:

```
sns.kdeplot(data, fill=True)
```

Histogram with Density Curve:

```
sns.histplot(data, bins=10, kde=True)
```

Solution Program

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Sample observed data
data = [45, 50, 52, 55, 58, 60, 62, 65, 68, 70,
        72, 75, 78, 80, 82, 85, 88, 90, 92, 95]

# Create DataFrame
df = pd.DataFrame({'Marks': data})

print("Observed Data:")
print(df)

# Create Histogram
plt.figure(figsize=(8, 5))
sns.histplot(df['Marks'], bins=6, kde=False)

plt.title("Histogram of Marks")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# Create Density Plot
plt.figure(figsize=(8, 5))
```

```
sns.kdeplot(df['Marks'], fill=True)

plt.title("Density Plot of Marks")
plt.xlabel("Marks")
plt.ylabel("Density")
plt.tight_layout()
plt.show()
```

Output

Observed Data:

```
    Marks
0      45
1      50
2      52
3      55
4      58
5      60
6      62
7      65
8      68
9      70
10     72
11     75
12     78
13     80
14     82
15     85
16     88
17     90
18     92
19     95
```

Graphical Output

1. Histogram:
Displays the marks divided into intervals (bins), with the height of each bar representing the **frequency** of observations in that interval.

2. Density Plot:
Displays a smooth curve representing the estimated **continuous probability distribution** of the observed marks.

3. Combined Histogram + Density Plot can also be generated using:

```
sns.histplot(df['Marks'], bins=6, kde=True)

plt.title("Histogram with Density Curve")
plt.xlabel("Marks")
plt.ylabel("Frequency / Density")
plt.show()
```

Explanation

- **Histogram** → shows how frequently values occur in different intervals.
- **Density plot (KDE)** → gives a smooth estimate of the underlying probability distribution.
- `bins=6` divides the data into six intervals.
- `kde=True` adds the density curve to the histogram.
- `fill=True` fills the area under the density curve.

Result

Thus, a histogram and density plot were successfully created to visualize the **frequency distribution** and **continuous probability distribution** of the observed data.

D). Create Scatter Plot and Examine the Relationship Between Two One-Dimensional Data Series

Aim

To create a **Scatter Plot** using two one-dimensional data series and examine the relationship or correlation between them using Python, Pandas, and Matplotlib.

Syntax

Matplotlib:

```
plt.scatter(x, y)
```

Pandas:

```
df.plot.scatter(x='Column1', y='Column2')
```

Solution Program

```
import pandas as pd
import matplotlib.pyplot as plt

# Two one-dimensional data series
study_hours = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
marks = [45, 50, 55, 60, 65, 70, 72, 80, 85, 90]

# Create DataFrame
df = pd.DataFrame({
    'Study_Hours': study_hours,
    'Marks': marks
})

print("Data:")
print(df)

# Create Scatter Plot
plt.figure(figsize=(8, 5))

plt.scatter(df['Study_Hours'], df['Marks'])

plt.title("Relationship Between Study Hours and Marks")
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.grid(True)

plt.tight_layout()
plt.show()

# Calculate correlation
correlation = df['Study_Hours'].corr(df['Marks'])
```

```
print("\nCorrelation Coefficient:", round(correlation, 2))
```

Output

Data:

```
   Study_Hours  Marks
0            1     45
1            2     50
2            3     55
3            4     60
4            5     65
5            6     70
6            7     72
7            8     80
8            9     85
9           10     90

Correlation Coefficient: 1.00
```

Graphical Output

The scatter plot contains:

- **X-axis:** Study Hours
- **Y-axis:** Marks
- Each point represents one observation.
- The points move upward as study hours increase, indicating a **positive relationship** between study hours and marks.

Explanation

A scatter plot is used to examine the relationship between two numerical variables.

- If points move **upward**, there is a positive relationship.
- If points move **downward**, there is a negative relationship.
- If points are randomly distributed, there may be little or no relationship.
- The correlation coefficient ranges from **–1 to +1**.
    - `+1` → perfect positive correlation
    - `0` → no linear correlation
    - `−1` → perfect negative correlation

In this example, the correlation coefficient is approximately **1.00**, showing a strong positive linear relationship between study hours and marks.

Result

Thus, the scatter plot was successfully created and the relationship between the two one-dimensional data series was examined using the correlation coefficient.

Create Box Plots to Visualize Data with Many Categorical Variables

Aim

To create **Box Plots** for visualizing the distribution of numerical data across multiple categorical variables using Python, Pandas, Matplotlib, and Seaborn.

Syntax

Seaborn:

```
sns.boxplot(x='Category', y='Value', data=df)
```

Pandas:

```
df.boxplot(column='Value', by='Category')
```

Solution Program

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Create sample data
data = {
    'Department': [
        'CSE', 'CSE', 'CSE', 'CSE', 'CSE',
        'ECE', 'ECE', 'ECE', 'ECE', 'ECE',
        'EEE', 'EEE', 'EEE', 'EEE', 'EEE'
    ],
    'Marks': [
        78, 85, 90, 72, 88,
        65, 70, 82, 75, 80,
        60, 68, 72, 76, 85
    ]
}

df = pd.DataFrame(data)

print("Data:")
print(df)

# Create Box Plot
plt.figure(figsize=(8, 5))

sns.boxplot(
    x='Department',
    y='Marks',
    data=df
)

plt.title("Marks Distribution by Department")
plt.xlabel("Department")
plt.ylabel("Marks")
```

```
plt.grid(axis='y', linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```

Output

```
Data:
   Department  Marks
0         CSE     78
1         CSE     85
2         CSE     90
3         CSE     72
4         CSE     88
5         ECE     65
6         ECE     70
7         ECE     82
8         ECE     75
9         ECE     80
10        EEE     60
11        EEE     68
12        EEE     72
13        EEE     76
14        EEE     85
```

Graphical Output

The box plot displays one box for each category:

- **CSE** → distribution of CSE marks
- **ECE** → distribution of ECE marks
- **EEE** → distribution of EEE marks

Each box plot shows:

- **Median** – middle value
- **Q1** – first quartile
- **Q3** – third quartile
- **IQR** – Q3 – Q1
- **Whiskers** – range of typical observations
- **Outliers** – unusually high or low observations

Explanation

A **Box Plot** is useful when a dataset contains a numerical variable and one or more categorical variables. It allows the distributions of several categories to be compared in a single graph.

For example, in this program, `Department` is the **categorical variable**, while `Marks` is the **numerical variable**.

The following statement creates the box plot:

```
sns.boxplot(x='Department', y='Marks', data=df)
```

Here:

- `x='Department'` places categories on the X-axis.
- `y='Marks'` places numerical values on the Y-axis.
- `data=df` specifies the DataFrame.

Result

Thus, the **Box Plot** was successfully created to visualize and compare the distribution of numerical data across multiple categorical variables.

5. Data Visualization with Matplotlib and Seaborn

a. Line Plot

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20, 25, 30]
plt.plot(x, y)
plt.title("Line Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.savefig("line_plot.png")
plt.show()
```

b. Bar Plots

```
import pandas as pd
df = pd.DataFrame({'Category': ['A', 'B', 'C'], 'Values': [10,
20, 15]})
df.plot(kind='bar', x='Category', y='Values')
plt.title("Bar Plot")
plt.show()
```

c. Histogram and Density Plot

```
import seaborn as sns
import numpy as np
data = np.random.randn(1000)
sns.histplot(data, kde=True)
plt.title("Histogram and Density")
plt.show()
```

d. Scatter Plot

```
x = np.random.rand(50)
y = np.random.rand(50)
plt.scatter(x, y)
plt.title("Scatter Plot")
plt.xlabel("x")
```

```
plt.ylabel("y")
plt.show()
```

e. Box Plot

```
sns.boxplot(data=np.random.randn(100, 4))
plt.title("Box Plot")
plt.show()
```